

A 0.078 pJ/SOP Unstructured Sparsity-Aware Spiking Attention/Convolution Processor with 3D Compute Array

Chaoming Fang¹, Ziyang Shen¹, Shiqi Zhao¹, Chuanqing Wang¹, Fengshi Tian², Jie Yang¹, Mohamad Sawan¹

¹Westlake University, Hangzhou, China; ²HKUST

Large-scale SNNs, which employ advanced network architectures such as Transformers, have shown performance matching that of their ANN counterparts [1]. The inherent high sparsity and the use of accumulation (AC) features in SNNs offer the promise of energy-efficient computing. However, the energy efficiency of SNNs is highly dependent on the data sparsity level and can deteriorate, falling even below the efficiency of resource-intensive ANNs when sparsity is low. Previous work [2] mitigates this problem by using heterogeneous SNN and CNN cores but incurs increased area and power consumption due to costly MAC array implementation. Therefore, a spiking-only accelerator with enhanced energy efficiency across all sparsity levels is highly desired. To achieve this goal, three critical challenges need to be addressed as shown in Fig. 1: 1) Redundant memory access of weights and partial sums across different time steps leads to significant power consumption and memory space waste. 2) Throughput degradation when exploiting unstructured spike sparsity. Fetching irregularly distributed non-zero spikes and corresponding weights one by one incurs long latency, compromising the benefits of high sparsity in large-scale SNNs. 3) One-size-fits-all scheduling leads to imbalanced efficiency across different operators. A homogeneous scheduler cannot be applicable to all operators at maximal efficiency because they have distinctive computational characteristics.

This work presents an unstructured sparsity-aware spiking attention and convolution accelerator for large-scale SNNs, as shown in Fig. 2. The processor has three key features: 1) A 3D computation array with timestep-paralleled adder trees to maximize computation throughput and on-chip data reuse across different time steps, reducing external memory access (EMA) by 68.3%. 2) A parallel non-zero data fetcher with a priority encoder array to generate addresses for fetching weights stored in multibank memories, corresponding to non-zero spikes. It transmits valid spike-weight pairs into the 3D array, resulting in an average computation latency reduction ranging from 32.3% to 84.7%. 3) A multi-mode unified computation scheduler that includes a sparse attention skipper to predict and skip zero self-attention output, an asynchronous data dispatcher for workload balancing among cores, and a bidirectional compressor for result compression in two directions to facilitate data transpose operations.

Fig. 3 shows the workflow of the processor and the data mapping strategy of the 3D computation array. Conventional dataflow computes states in each timestep sequentially, requiring repeated load of W and store of neuron state V_t . This processor adopts a parallel timestep accumulation dataflow. It first calculates the accumulated states in multiple timesteps separately in parallel, and then the state dependencies of each timestep are handled sequentially but its latency can be hidden in pipeline. The above dataflow is implemented on a 3D computation array with data reuse exploited in all three dimensions. Weight reuse occurs in the depth direction, allowing parallel computation with spikes from multiple timesteps. Spikes are reused vertically and $psums$ are reduced horizontally like conventional 2D arrays. State dependencies are handled with the neuron state update circuit. It compares V in each timestep with threshold V_T , and determines to add V_{t-1} or V_{rst} to V_t . The proposed approaches significantly reduce data movement of weights and partial sums across different time steps, resulting in a 68.3% reduction in total EMA compared to a baseline 2D array.

Fig. 4 depicts the structure of the parallel non-zero data fetcher. Spike data are compressed into a global bitmap plus a local bitmap. The former one represents which input channels has all-zero timesteps, and the latter one represents spike information in the remaining channels. Weights are stored in separate memory banks according to their input-channel indexes. Sixteen 8-input priority encoders are implemented to generate fetching addresses for weight SRAM banks in parallel, using the global spike bitmap as input. This enables the fetching of eight irregularly distributed weight vectors every cycle, taking advantage of both compute parallelism and

sparsity. To prevent underutilization of hardware with idle input ports in the adder tree, a load balancer is implemented. It allows an empty channel to borrow weight values from non-zero channels within a given size of a neighboring window. To prevent conflicts, each channel in the neighbor search window is assigned a priority code, ensuring that a non-zero weight will be allocated to the channel with the highest assigned priority. Increasing the search window size reduces average latency, but it also introduces logic complexity, leading to higher power and area consumption. Experimental results indicate that a search size of 4 achieves the best balance between latency improvement and resource overhead. Under typical sparsity levels, the implementation of sparse exploitation techniques reduces average latency by up to 84.7% compared to a dense dataflow.

Fig. 5 illustrates a multi-mode computation scheduler. In spiking convolution, the fan-in is usually high and thus, input spikes are allocated to multiple cores to accommodate the high fan-in level. A memory router between cores enables spatial reuse of input spikes by allowing a core to fetch spike data from its neighboring core instead of repeatedly fetching from the spike memory. Asynchronous data dispatcher is implemented to enable each core to continue computing the next position without waiting for other cores to finish, reducing processing latency by 8.1% compared to synchronous scheduling. For QKV generation, the scheduler incorporates a bidirectional compressor to handle data transpose in K . The compression direction for each output tensor of the computation array should align with its fan-in direction of the subsequent spiking self-attention (SSA) computations. Horizontal mode compresses an output tensor along its generated direction d , which is suitable for compressing Q . However, for K^T and V , vertical compression is performed using output from multiple cycles, as the data generation direction (d) does not align with their SSA fan-in direction N . The bidirectional compressor ensures proper storage format in memory and eliminates the need for additional buffers to transpose the matrix before SSA computation. In SSA, the processor maps K^T in the spike memory, V in the weight memory, and one line from Q in the sparse attention skipper. The global bitmap of Q controls the fetching of a line of K^T from the spike memory, while the local bitmap of Q performs a bitwise AND operation with K to control the input of the adder tree. The sparse attention skipper achieves an average operation reduction of 78% in SSA computation.

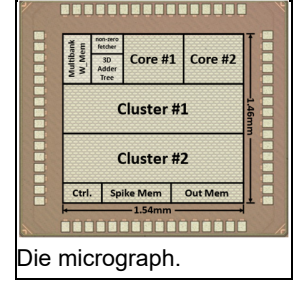
Fig. 6 shows the experimental results of the accelerator. Fabricated in 40nm CMOS technology, the accelerator can operate with a supply voltage of 0.56-1.1V and a working frequency of 50-200 MHz. We evaluated the efficiency of processor with TOPS/W and pJ/SOP. 20Ps represents 8 accumulations in our work because it consumes similar energy to one 8b-MAC. Consistent with [2], pJ/SOP is defined as the energy consumed by each spike operation. In a sparsity range of 0%-97%, the accelerator demonstrates an energy efficiency of 20.3-132.4 TOPS/W, outperforming the ANN baseline across all sparsity levels [2]. Benchmarked with ImageNet and CIFAR-10 image recognition and DVS128 hand gesture recognition, this work achieves the highest recognition accuracy in all tasks compared to prior works, thanks to its support for spiking transformers. Compared to previous SNN processors [2-4] or sparse ANN accelerators [5, 6], this work achieves the best overall energy efficiency of 42.8 TOPS/W and 0.078 pJ/SOP at ImageNet, which is 3.57x higher than the SOTA sparse ANN processor [6], and also 77.7% lower than the SOTA SNN processor [3]. Meanwhile, the task energy for ImageNet and CIFAR-10 is 2.3x and 9.3x lower than [3].

Acknowledgement:

Authors acknowledge support from Westlake University, grant number 10318A992001, and from STI2030-Major Projects 2022ZD0208805. The corresponding authors of this paper are Jie Yang and Mohamad Sawan.

References:

- [1] Z. Zhou *et al.*, "Spikformer," ICLR, 2023
- [2] S. Kim, *et al.*, "C-DNN," ISSCC, 2023.
- [3] S. Kim, *et al.*, "SNPU," JSSC, 2023.
- [4] J. Zhang, *et al.*, "ANP-I," ISSCC, 2023.
- [5] S. Moon, *et al.*, "Arbitrarily Quantized," ISSCC, 2023.
- [6] S. Sun, *et al.*, "2D/3D Unified," ISSCC, 2023.



Die micrograph.

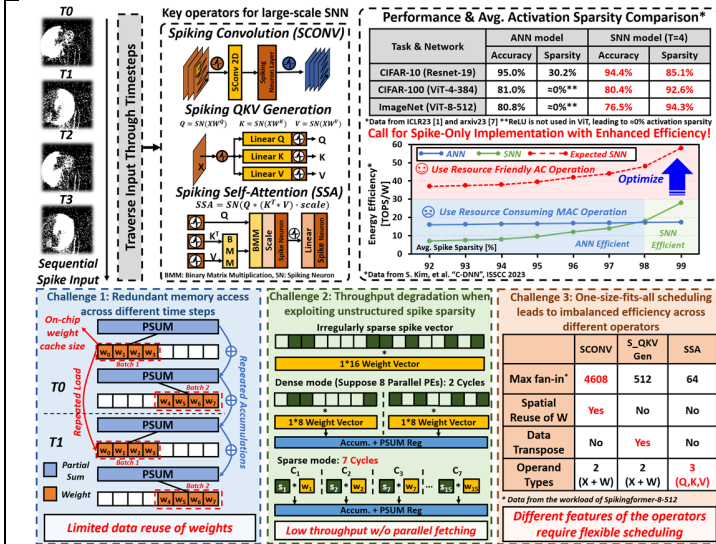


Fig. 1. Key operators and challenges for energy-efficient large-scale spiking neural network.

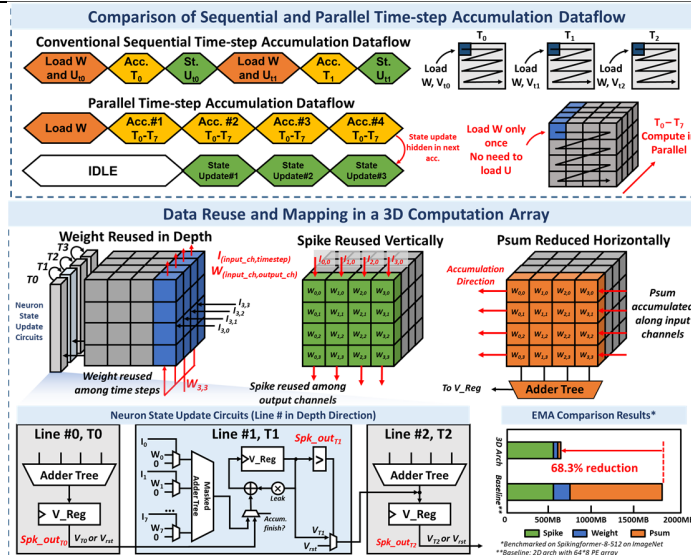


Fig. 3. Parallel time-step accumulation dataflow and three types of data reuse in 3D computation array to reduce memory access.

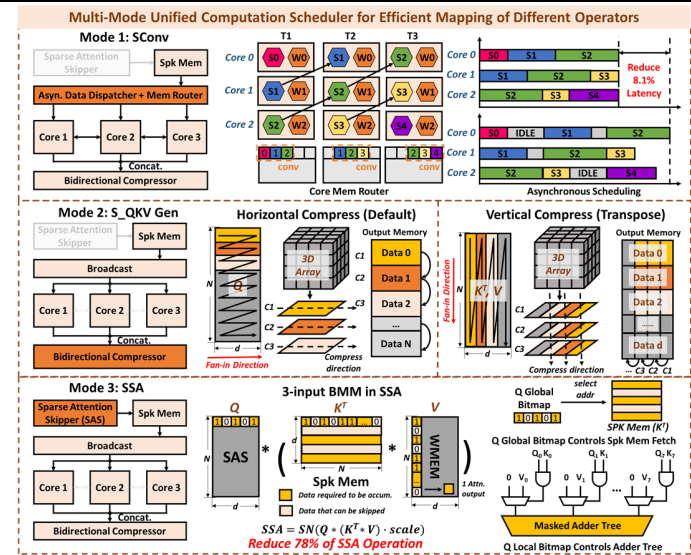


Fig. 5. Multi-mode unified computation scheduler to optimize dataflow of SCONV, S QKV Gen, and SSA.

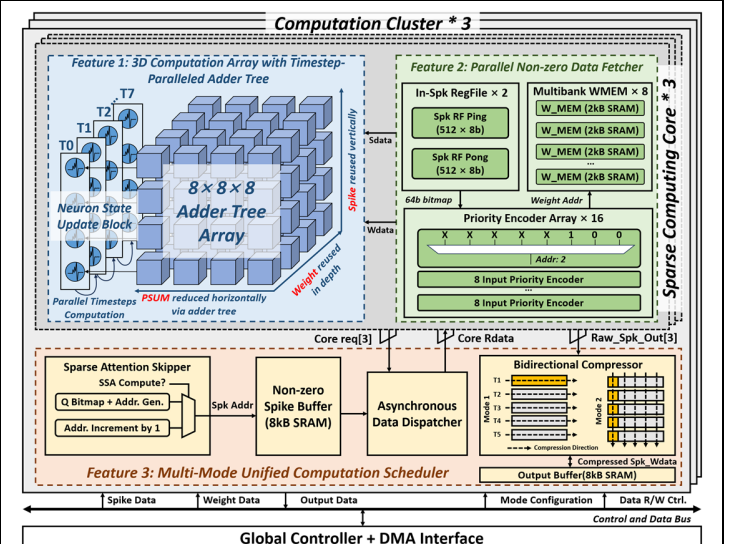


Fig. 2. Overall architecture and key features for the unstructured sparsity-aware SNN accelerator with 3D computation array.

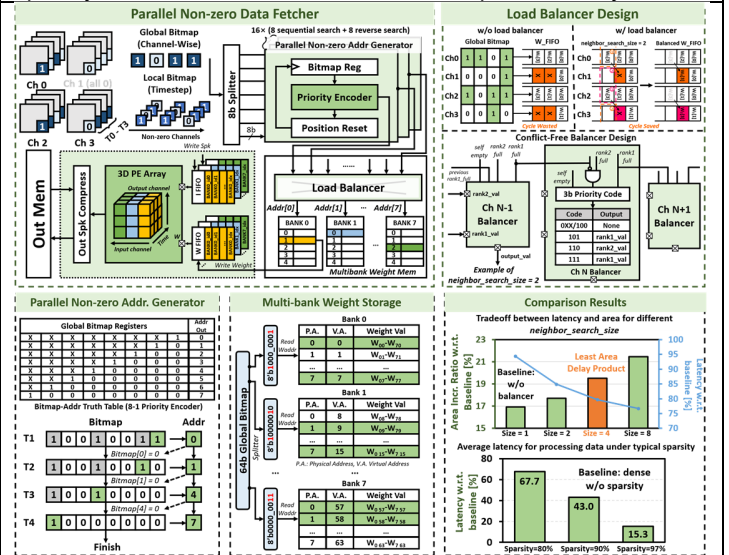


Fig. 4. Parallel non-zero data fetcher with priority encoder and load balancer for high throughput sparse computing.

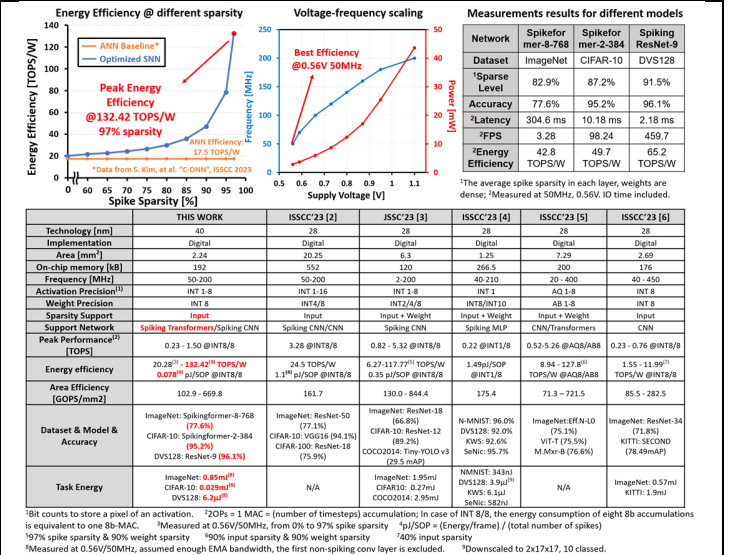


Fig. 6. Measurement results and comparison table of SNN processors and sparse accelerators.